ITERATIONS & CONDITIONAL CHECKS

I find loops and if conditions the hardest to get working. To begin with, I wanted loop / if conditions to be handled by the run time stack. I was unable to get nested loops / if conditions to work. So I decided to use a different approach.

I would use an open brace to mark the start of the loop / if condition and a close brace to mark the end of this code block, similar to 'C' function.

```
{ code ... } 10 int 20 int <if "it ended" print-str
```

This means wherever I see a '{', a new function to handle this code is called, and space on the stack is reserved for 4 important parts.

```
sub   rsp, 0x04 * 0x08
```

#1 If the loop or if condition are true, where does the processing begin. This is found straight after the '{'

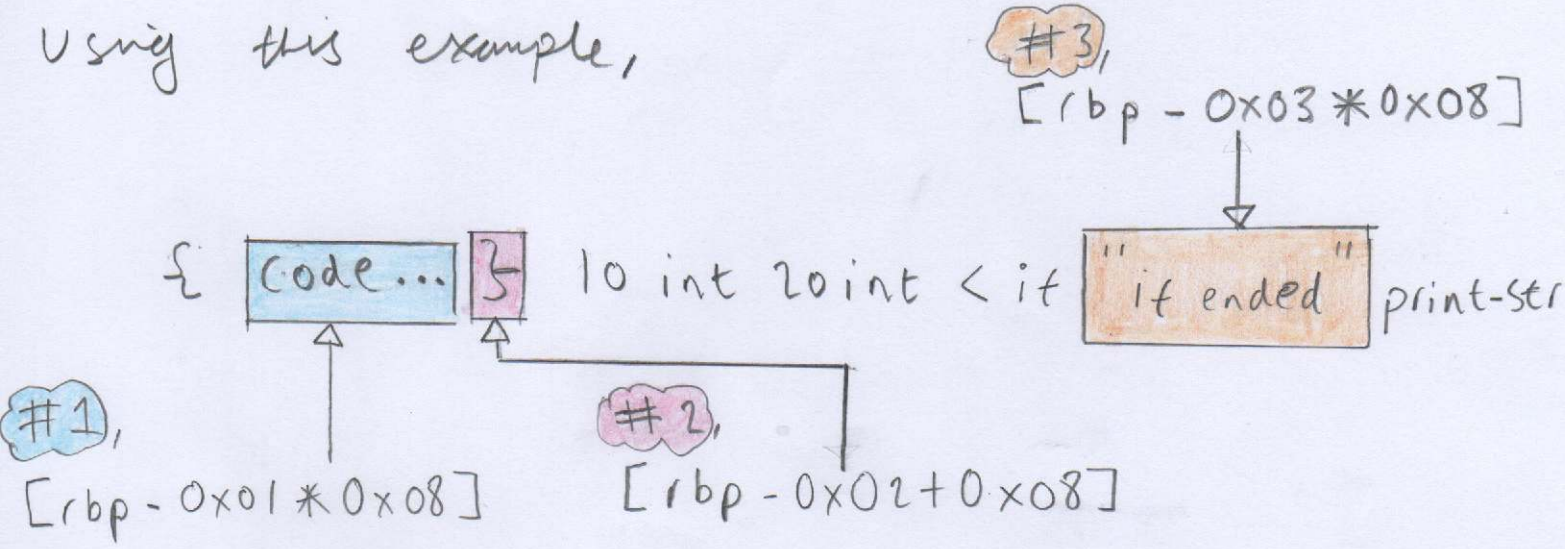
#2 Where the '}' close brace is found

#3 Where to resume once the if / loop has ended. This will be the token after the if / loop operand.

#4 A flag to set whether we repeat the above steps (if it's a 'loop') or to end the function if it was an 'if' statement.

Once I've put all the information in the stack, I use [rbp - n * 0x08] to access the part I need, where n is 1...4.

Using this example,



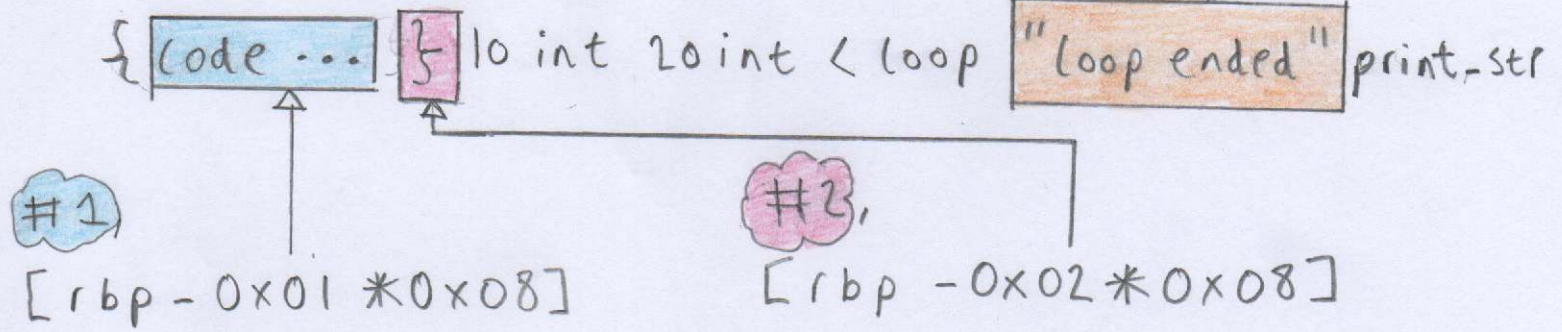
#4,
[rbp - 0x04 * 0x08], is set to 0 because it isn't a loop.

Using this example, when '{' is found, we enter a function. We grab the first token and store this. We get the matching '}', this is also saved. The expression after the '}' is now read. This should leave a 0 (false) or 1 (true) on the top of the run time stack. Once we see our if / loop operand, we grab the next token after this. We also set the loop flag to 0 or 1 depending on whether it's a if / loop statement.

The expression '10 int 20 int <' leaves a 1 on the top of the run time stack. The 'if' operand checks the top of the run time stack and jumps to the token after the '{'. If the top of the run time stack was 0, then we jump to the token after the if operand and ignore the if statement.

In this case the code between the curly braces is read. When we use the '}' brace, the expression is NOT read again and we jump to the token after the if operand, because we have finished with the if statement.

Using the same example in a loop, #3, [rbp - 0x03 * 0x08]



#1 [rbp - 0x01 * 0x08]

#2, [rbp - 0x02 * 0x08]

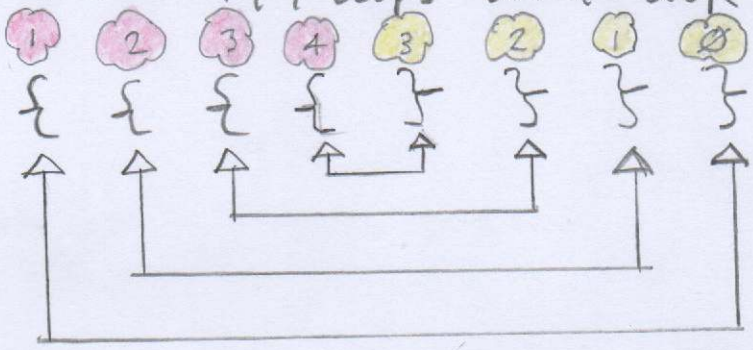
#4, [rbp - 0x04 * 0x08], is set to 1 because it is a loop.

The same steps happen here, apart from the flag being set to 1 because this is a loop.

That means the expression '10 int 20 int <' is true (1) and the loop operand will jump to the first token after the '{'. Once the '}' is reached, I check the loop flag and it's 1 (true). That means I read the expression '10 int 20 int <' and repeat the same steps. This is an example of an infinite loop that will go on forever!

I could change this by using a variable within the check condition to break the loop.

After the if / loop has ended, the function is closed and the stack is cleared up. This also lets me run nested if / loop statements with no problems. The braces for nested if / loops could look like this:



To make sure the braces match a simple function was made that starts a register with value 1. When another open bracket is matched, the register is ++ and when a closed bracket is matched, the register is --. When the register is 0, thats the matching close bracket.